

Training Neural Nets using only an Approximate Tableless LNS ALU

Mark Arnold
XLNS Research, USA
markarnold@xlnsresearch.com

Ed Chester
Goonhilly Earth Station, U.K.
ed.chester@goonhilly.org

Corey Johnson
XLNS Research, USA
coreydavidjohnson1995@gmail.com

Abstract—The Logarithmic Number System (LNS) is useful in applications that tolerate approximate computation, such as classification using multi-layer neural networks that compute non-linear functions of weighted sums of inputs from previous layers. Supervised learning has two phases: training (find appropriate weights for the desired classification), and inference (use the weights with approximate sum of products). Several researchers have observed that LNS ALUs in inference may minimize area and power by being both low-precision and approximate (allowing low-cost, tableless implementations). However, the few works that have also trained with LNS report at least part of the system needs accurate LNS. This paper describes a novel approximate LNS ALU implemented simply as logic (without tables) that enables the entire back-propagation training to occur in LNS, at one-third the cost of fixed-point implementation.

Index Terms—approximate computation; logarithmic arithmetic; deep learning; neural networks; back-propagation

I. LOGARITHMIC NUMBER SYSTEM

The Logarithmic Number System (LNS) [20] is similar to Floating Point (FP). Early LNS [20] used Read Only Memories (ROMs) to look up “ideal” LNS values for modest wordsize. Larger wordsizes need extra hardware to make LNS accurate [6]. Instead, this paper describes low-cost, approximate LNS for back-propagation training. A real number X can be represented in LNS with a tuple $x = (x_L, x_S)$ as:

$$\begin{aligned} x_S &= \text{sign}(X) \\ x_L &= x_I + x_F \approx \log_b |X| + \Omega \end{aligned} \quad (1)$$

where b is the base of the logarithm, Ω is a constant offset, x_S is the sign of the real, x_L is a fixed-point approximation of its logarithm consisting of x_I (an integer portion) and x_F (a fractional portion). LNS offers advantages for applications with two properties. First, most numbers need a limited range:

$$X_{\min} \leq |X| < X_{\max}, \quad (2)$$

where the usual assumption is symmetrical two’s-complement binary ($\Omega = 0$) for x_L leading to $X_{\max} \approx 1/X_{\min}$. The application tolerates occasional $|X|$ outside (2) being clipped. Also, subnormals are clipped. Second, the application tolerates modest relative precision with an F -bit fractional part, x_F , for a wordsize $W = 2 + K + F$ including the value sign, x_S (is $X < 0.0?$) and the logarithm sign (is $|X| < 1.0?$).

LNS does not require b to be an integer, and selecting $b > 2$ reduces the precision and increases the range. LNS gives the designer a choice of range/precision tradeoffs for each W .

Most earlier LNS neural nets [2], [3], [7]–[9], [12]–[14], [16], [18], [19] use $b = 2$; however [5] suggests $b = 2.19$ improves back-propagation training.

LNS products, quotients and powers are exact. For example, to divide X by Y giving Z , an LNS ALU receives $x = (x_L, x_S)$ and $y = (y_L, y_S)$ and in response computes $z_S = x_S \oplus y_S$ and $z_L = \text{clip}(x_L - y_L + \Omega)$, where $\text{clip}(\cdot)$ limits the result to be within the interval (2). LNS sums use the addition-logarithm, $s_b(z_L) = \log_b(1.0 + b^{z_L})$, which is analogous to $1.0 + |Z|$. To add two positive real numbers, LNS uses $r_L = \text{clip}(y_L + s_b(x_L - y_L))$. This is like manipulating the value conventionally as $R = Y + X = Y(1 + X/Y)$. We may assume $z_L \leq 0$ because $X + Y = Y + X$, and choose

$$r_L = \text{clip}(\max(x_L, y_L) + s_b(-|x_L - y_L|)). \quad (3)$$

Analogous for differences, $d_b(z_L) = \log_b |1 - b^{z_L}|$, which has a singularity at $d_b(0)$ that makes this difficult to approximate. The ALU selects whether to compute s_b or d_b based on the signs, x_S and y_S . The hardware uses s_b when the signs are the same and d_b when the signs are different.

All earlier LNS neural-net implementations assume $d_b(0)$ should return $x_{\min} = \log_b(X_{\min})$; this paper takes a different approach that reduces hardware cost without raising serious issues for back-propagation. Recent attempts with LNS back-propagation [5], [18] have reported success using crude approximations for s_b , especially in the hidden layer; however, d_b is more problematic, especially when feeding back the difference between the actual output layer and the desired classification. For this reason [5], [18] report using relatively large (over 1Kbit) ROMs for d_b , at least in the output layer (or d_b in all layers [5]). This paper describes the first approximate LNS ALU that can implement back-propagation successfully without needing any tables for either s_b or d_b .

II. MITCHELL’S METHOD

Mitchell’s [15] approximations for $\log_2(X)$ and 2^{x_L} have been used to approximate products in feed-forward inference by various neural-network designs [7]–[9], [12]–[14]. In contrast, this paper uses only Mitchell’s antilogarithm approximation, $M_F(z_L)$, in a different way during LNS addition. The idea is based on $2^{z_F} \approx M_F(z_F) = 1 + z_F$ for $0 \leq z_F \leq 1$. Arguments outside this range yields can be shifted

$$M_F(z_L) = 2^{z_I} \cdot (1 + z_F). \quad (4)$$

A cruder, truncated Mitchell approximation [1] retains only the high-order J bits of z_F , which will be referred to as z_J :

$$M_J(z_L) = 2^{z_I} \cdot (1 + z_J). \quad (5)$$

III. LNS ADDITION WITH MITCHELL'S METHOD

The first attempt to exploit Mitchell's method for LNS addition [4] computed $s_2(z_L) \approx M_F(z_L)$, for $z_L \leq 0$, using the untruncated Mitchell approximation. Doing this for base-two seems natural since Mitchell's method was originally defined for $b = 2$ logarithms. Recently, the concept of selecting $b > 2$ when computing s_b with Mitchell's method was shown to improve LNS back-propagation [5]. The Mitchell hardware manipulates the bits in the same way regardless of b , but the application's values are converted in and out of the hardware slightly differently, which impacts the cumulative errors.

A cheaper but cruder approach for back-propagation was suggested by Sanyal et al. [18] that approximates $s_2(z_L) \approx M_0(z_L)$, although it was labeled as a "bit-shifting" method taken from [14], rather than as a truncated Mitchell method. One novel contribution of this paper is to recognize there is a range of truncated Mitchell approximations $s_b(z_L) \approx M_J(z_L)$ that trade cost and accuracy. By keeping a small number (perhaps $J = 2$) bits in the Mitchell shifter, the hardware cost is reduced from [5] with better accuracy than [14], [18].

IV. LNS SUBTRACTION WITH MITCHELL'S METHOD

The unsolved problem in [2], [18] is tableless LNS subtraction. Sanyal et al. [18] describe bit shifting of $1.5_{10} = 1.1_2$, but this is not accurate enough for the output layer. Instead, $d_b(z_L) \approx -M_J(z_L + c)$, $c \approx 0.375$, yields a tableless ALU:

$$\begin{aligned} r_S &= \begin{cases} x_S & x_L > y_L \\ y_S & x_L \leq y_L \end{cases} \\ r_L &= \end{aligned} \quad (6)$$

$$\begin{cases} \text{clip}(\max(x_L, y_L) + M_J(-|x_L - y_L|)) & x_S = y_S \\ \text{clip}(\max(x_L, y_L) - M_J(-|x_L - y_L| + c) - \gamma) & x_S \neq y_S \end{cases}$$

where γ is a singularity correction factor (discussed in section V). For the moment, we let $\gamma = 0$. Figure 1 shows the exact $s_b(z)$ for $b = 2$ (green) and $b = 2.19$ (blue) along side its untruncated (red) and truncated (magenta) Mitchell approximations, $M_F(z_L)$ and $M_J(z_L)$, resp. The exact $s_2(z_L)$ is greater than or equal to the $M_F(z_L)$ Mitchell approximation except at 0 and $-\infty$. Generally speaking $s_{2.19}(z_L)$ is closer and less than the Mitchell approximation, which is why [5] suggests using this base. Using M_J actually is on average closer to $s_{2.19}(z_L)$ than $M_F(z_L)$. Figure 1 also shows the exact $d_b(z)$ for $b = 2.19$ (yellow) next to its untruncated (cyan) and truncated (black) Mitchell approximations, $-M_F(z_L + c)$ and $-M_J(z_L + c)$, resp. For $z_L < -1$, the Mitchell approximations are reasonably close to the exact d_b . The error becomes significant for $-0.5 \leq z_L < 0$ (Figure 1 does not show the full extent of the exact singularity below -1.5), but the proposed approximation here is better than the constant -1.5 provided by [18]. The next section considers how this impacts back-propagation performance, and describes how γ may be used to improve this.

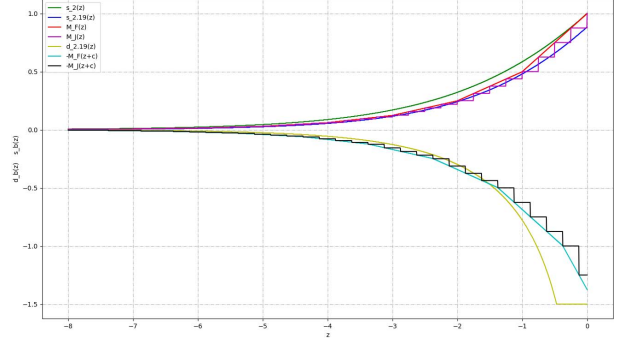


Fig. 1. Exact versus Mitchell Approximations, $F = 8$, $c = 0.375$, $\gamma = 0$.

V. BACK-PROPAGATION EXPERIMENTS

We experiment with truncated Mitchell LNS addition (6) in classical back-propagation learning [17] for a feed-forward neural network with one hidden layer. The weighted sum of inputs in each layer is used to compute an activation function, which might be one of several options. The sigmoid, $f(x) = 1/(e^x + 1)$, was a common choice in early back-propagation literature (including LNS experiments [2], [3], [5]), but it has the problem of vanishing gradients where training may get "stuck". Many modern deep-learning algorithms [11] opt for simpler activation functions, such as Leaky Rectified Linear Unit (LReLU), that was used in Sanyal's [19] code:

$$f_1(x) = \begin{cases} \beta \cdot x & x < 0 \\ x & x \geq 0 \end{cases}, \quad (7)$$

where β is a small constant. Classification using one-hot labels [10], [19] uses the softmax function for the output layer:

$$f_2(\mathbf{x}) = \frac{e^{\mathbf{x}_i}}{\sum_{j=1}^n e^{\mathbf{x}_j}}, \quad 1 \leq i \leq n. \quad (8)$$

To experiment with Mitchell LNS, we start with Sanyal's floating-point¹ back-propagation code [19]. We developed our own overloaded class for LNS that implements (5), (6), etc. We use two problems for training experiments: a) XOR, a trivial-sized (4 exemplars with two binary inputs and one binary output) but highly non-linear set which has difficulty converging, even when implemented with double floating-point [17]; and b) MNIST, a modestly large (thousands of exemplars with 28×28 gray-scale images of handwritten digits having ten one-hot-labeled outputs) which allows testing of the implementations' abilities to generalize [10].

XOR was an LNS example in [2], [5] and here we reproduce and extend those results on a neural network with two hidden units, sigmoid activation and weight matrices of size 3×2 and 3×1 . Arnold et al. [5] report around 75% convergence (the same as FP) with Mitchell- s_b and table-based d_b . Here, we achieve almost this convergence (70%) for symmetrical ($W = 14$, $F = 8$, $\Omega = 0$, $\gamma = 0$) and asymmetrical ($W = 13$, $F = 8$, $\Omega = 11$, $\gamma = 0$) tableless LNS ALUs (6).

¹At the time of this writing, [19] has no LNS code for back propagation.

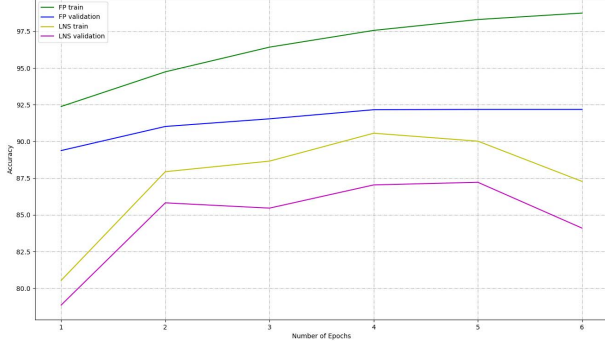


Fig. 2. FP vs. $F = 8, \gamma = 0$ Mitchell LNS, 5000 MNIST instances.

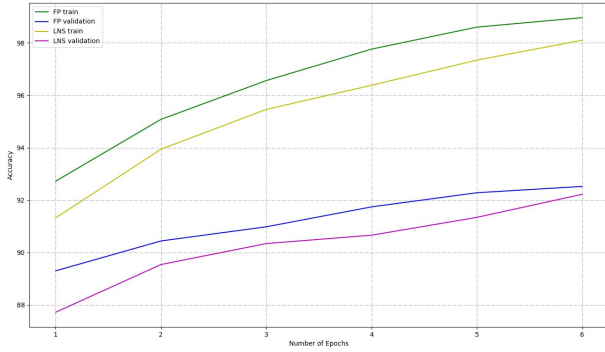


Fig. 3. FP vs. $F = 8$ Ideal LNS, 5000 MNIST instances.

Sanyal et al. [18] report that MNIST performance with 16-bit LNS is similar to 16-bit fixed point using 100 hidden units, f_1/f_2 activation and weight matrices of size 785×100 and 101×10 . Figure 2 shows the performance of 5000 MNIST exemplars using the tableless LNS ALU (6) with $F = 8$, $\gamma = 0$, $c = 0.359$, $J = 2$. LNS is never as good as FP (LNS training worse than FP validation), and after six epochs LNS performance begins to diminish, rather than converging to be similar to FP (as was reported in [18]). Much better performance (LNS training better than FP validation) with the same exemplars occurs in Figure 3 ($F = 8$ table-based s_b and d_b). We conjecture Figure 3 is like [18]: able to do better because of table-based ALUs for the output layer.

Rather than resort to tables, we hypothesize that the problems in Figure 2 with $\gamma = 0$ as well as those in [5], [18] stem from not capturing the shape of the singularity of d_b , allowing miscalculation of the difference between the actual output and the desired training output during back-propagation. As the LNS weights start to get close to their desired configuration, such miscalculation is fed back to the weights, wrongly nudging the weights away from convergence, as Figure 2 shows at epoch 6. We propose a solution by

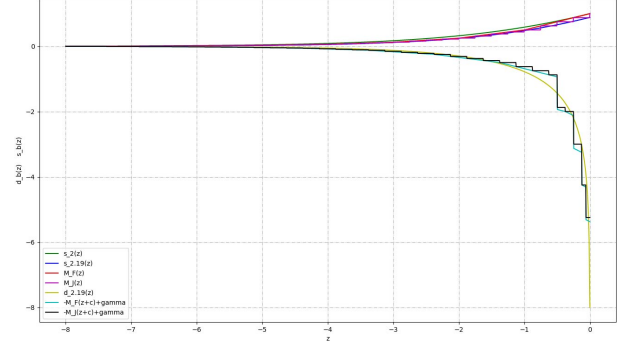


Fig. 4. Mitchell Approximations: $F = 8$, $c = 0.375$, γ as (9).

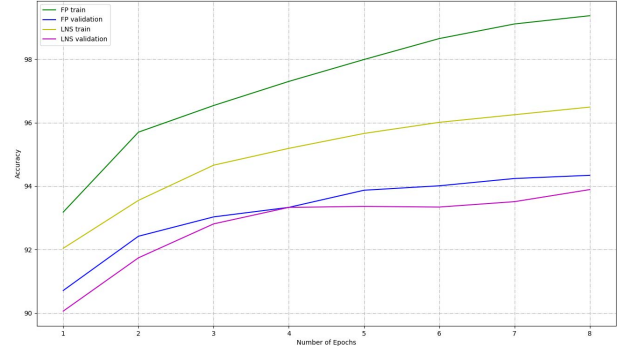


Fig. 5. FP vs. $F = 10$, $J = 4$ Mitchell LNS, $\gamma=9$, 10000 MNIST instances.

computing the singularity correction factor as

$$\gamma = \begin{cases} 1 & -2^{-2} \geq -|x_L - y_L| > -2^{-1} \\ 2 & -2^{-3} \geq -|x_L - y_L| > -2^{-2} \\ 3 & -2^{-4} \geq -|x_L - y_L| > -2^{-3} \\ 4 & 0 \geq -|x_L - y_L| > -2^{-4} \\ 0 & \text{otherwise,} \end{cases} \quad (9)$$

which crudely tracks the sudden change $d_b(z_L)$ experiences as $|x_L - y_L| \rightarrow 0$, as illustrated in Figure 4. In theory, γ should be > 4 for values closer to zero (becoming $-x_{\min}$ at zero), but in a compromise between between cost and accuracy we let results $0.0 \leq |R| < b^{-4}$ be represented by $r_L \approx -4$ in the hope such a slightly overstated nudge is not enough to cause serious disruption to the weights. Quantizing (9) as an integer (rather than the full precision d_b as would be obtained from a table) reduces the cost of adding $-\gamma$ to the final result.

Figure 5 shows the performance with 10000 MNIST exemplars using γ as given in (9) and $F = 10$, $c = 0.375$, $J = 4$. Although there is some difference with the training accuracy, the validation accuracy (on 10000 separate images) for LNS and FP are nearly identical. Since inference accuracy is what matters to the practicality of LNS, this suggests a $W = 16$ tableless LNS ALU (6) can implement the complete back-propagation algorithm, which is an improvement over Sanyal et al's $W = 16$ LNS requiring tables for the output layer [18].

TABLE I
SYNTHESIS RESULTS FOR VARIOUS MITCHELL LNS ALUS

W	F	J	c	LUTs	
				$\gamma = (9)$	$\gamma = 0$
14	8	stub		79	
14	8	2	0.375	104	85
14	8	8	0.375	104	92
15	9	stub		77	
15	9	2	0.375	105	86
15	9	9	0.375	110	95
16	10	stub		72	
16	10	2	0.375	110	92
16	10	10	0.375	110	99

VI. HARDWARE SIMULATION AND SYNTHESIS

A truncated Mitchell LNS ALU (6) combined with an LNS multiplier were implemented in behavioral Verilog. We replicated XOR training with a Verilog testbench, producing effectively identical results (given the same initial weights). We synthesized this using Vivado on a Xilinx Artix-7 FPGA without datapath or timing constraints for various values of F , J , c and γ as listed in Table I. These LNS multiply-accumulate circuits were implemented with only the FPGA's 'Look-Up Table' (LUTs) (no block RAMs or DSPs).

For comparison, a 'stub' LNS ALU that replaces s_b and d_b with identity functions gives an estimate of the baseline resources required to compute $\max(x_L, y_L)$, $-|x_L - y_L|$ and z_S . The differences between the LUTs column for $\gamma = 0$ and the stub give an estimate of the cost of the Mitchell shifter, which adds between between 6 (for $W = 14, J = 2$) and 27 LUTs (for $W = 16, J = 10$) overhead compared to what is minimally required to compute $\max(x_L, y_L)$, $-|x_L - y_L|$ and z_S . Using (9) requires between 24 (for $W = 14, J = 2$) and 38 (for $W = 16, J = 10$) LUTs compared to the stub.

Modern FPGAs have optimized DSP (fixed-point multiply-accumulate) units that map easily into the forward propagation of neural networks. In order to compare LNS results in Table I directly to a fixed-point equivalent in terms of LUTs, we synthesized a gate-level Verilog description of a fixed-point multiply-accumulate that requires 366 LUTs. In other words, in a head-to-head comparison, LNS takes one-third the resources. However, a practical designer might just let the synthesis tool use a single DSP unit.

Therefore, a more useful comparison of the proposed LNS ALU (6) against fixed-point involves comparing equivalent gate-level implementations as might be done in VLSI. For this, we replaced the behavioral Verilog with a hand-crafted gate-level netlist that implements the same algorithm (6) for $W = 14, F = J = 8, \gamma = 0, \Omega = 0$ using two-input gates. Table II compares this LNS multiply-accumulate circuit against our fixed-point multiply-accumulate circuit implemented using the same type of gates. As with FPGAs, the LNS circuit is about one third the complexity of the equivalent fixed-point circuit.

TABLE II
COMPARATIVE HARDWARE COST OF LNS VS. FIXED-POINT

	and	or	xor	mux	Total
LNS	166	94	143	41	812
Fixed	592	317	589	—	2676

VII. CONCLUSIONS

We have shown that a tableless LNS ALU based on novel improvements to Mitchell's method allows implementation of the complete back-propagation algorithm using one third of the resources required for a conventional fixed-point ALU. This suggests other algorithms, like convolutional neural networks [7], [11] may benefit from the techniques proposed here, but may also require singularity treatment more costly than (9).

REFERENCES

- [1] Khalid H. Abed and R. E. Siferd, "VLSI Implementation of a Low-Power Antilogarithmic Converter," *IEEE Trans. Comput.*, vol. 52, no. 9, pp. 1221–1228, Sept. 2003.
- [2] M. Arnold, et al., "Implementing Back-Propagation Neural Nets with Logarithmic Arithmetic," *Proc. Int. AMSE Conf. Neural Networks*, vol. 1, pp. 75–86, San Diego, May 1991.
- [3] M. Arnold, et al., "On the Cost Effectiveness of Logarithmic Arithmetic for Back-Propagation Training on SIMD Processors," *Proc. Int. Conf. Neural Networks*, Houston, vol. 2, pp. 933–936, June 1997.
- [4] M. Arnold, "LPVIP: A Low-power ROM-Less ALU for Low-Precision LNS," *14th Int. Workshop PATMOS*, LNCS 3254, pp. 675–684, Santorini, Greece, Sept. 2004.
- [5] M. Arnold, E. Chester, J. Cowles and C. Johnson, "Optimizing Mitchell's Method for Approximate Logarithmic Addition via Base Selection with Application to Back-Propagation," *IEEE NORCAS*, Helsinki, Oct. 2019.
- [6] H. Fu, et al., "FPGA Designs with Optimized Logarithmic Arithmetic," *IEEE Trans. Comput.*, vol. 59, no. 7, pp. 1000–1006, July 2010.
- [7] M. S. Kim, et al., "Efficient Mitchell Approximate log Multipliers for Convolutional Neural Networks," *IEEE Trans. Comput.*, vol. 68, no. 5, pp. 660–675, May 2019.
- [8] I. Kouretas and V. Paliouras, "Logarithmic number system for deep learning," *7th Int. Conf. MOCAST*, Thessaloniki, 2018, pp. 1–4.
- [9] J. Johnson, "Rethinking floating point for deep learning," *CoRR*, vol. abs/1811.01721, 2018.
- [10] Y. LeCun, "The MNIST Database of Handwritten Digits", <http://yann.lecun.com/exdb/mnist/>
- [11] Y. LeCun, Y. Bengio and G. Hinton, "Deep Learning", *Nature*, vol. 521, pp. 436–444, May 2015. doi: 10.1038/nature14539
- [12] P. Lee, et al., "LogTOTEM: A Logarithmic Neural Processor and its Implementation on an FPGA Fabric," *IJCNN*, Orlando, pp. 2764–2769, Aug. 2007.
- [13] E. H. Lee, et al., "Lognet: Energy-efficient neural networks using logarithmic computation", *ICASSP*, March 2017, pp. 5900–5904.
- [14] D. Miyashita, et al., "Convolutional Neural Networks using Logarithmic Data Representation," *arXiv:1603.01025*, Mar 2016.
- [15] J. Mitchell, "Computer Multiplication and Division using Binary Logarithms," *IEEE Trans. El. Comput.*, vol. EC-11, pp. 512–517, Aug. 1962.
- [16] S. Richfield "A Logarithmic Vector Processor for Neural Net Applications," *IEEE First International Conference on Neural Networks*, pp. 22–28, June 1987.
- [17] D. E. Rumelhart, et al., "Learning Internal Representations with Error Propagation," *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, MIT Press, pp. 345–355, 1986.
- [18] A. Sanyal, P.A. Beerel, K.M. Chugg, "Neural Network Training with Approximate Logarithmic Computations," *ICASSP*, May 2020, *arXiv:1910.09876*
- [19] A. Sanyal, "Deep Neural Networks multi-layer perceptron implementation using Logarithmic Number System," <https://github.com/arnabsanyal/lnsdnn.git>
- [20] E. E. Swartzlander and A. G. Alexopoulos, "The Sign/Logarithm Number System," *IEEE Trans. Comput.*, vol. C-24, pp. 1238–1242, Dec. 1975.